NPLify · P0 Technical Baseline

Transaction Lifecycle & Approval State Machines

New XP Technologies Limited

28 October 2026 · Confidential

How to read this pack

Four map pages are now part of the pack as documents of their own, and are the place to start: the Top-Down Map (from the business, through the data model and the deal states, to the scenarios and the reading order), the Deal Flow on one page (Quote, Collect, Convert, Settle, with the roles and the money path), the Workflow Tree (8 classes, 27 sub-classes, 93 procedures, each naming who initiates, who approves, the codes it is written under and the records it touches) and the Workflow Tree on one page. Read them in that order, then this document. Every worked transaction (T), acceptance vector (L, B), walkthrough (W, X), flow (F) and procedure (C, A, D) has one identifier, listed in the Reading Companion's identifier register; the Test Strategy & Coverage document says which test proves each procedure.

1 · Three roles, one principle

Every controlled action is **maker-checker**: one person initiates, a different person in the approving role approves, and the system refuses an approval by the initiator. Since the rights are configuration, not code: `ROLE` holds the three roles of today and `ROLE_RIGHT` says per controlled action which role may initiate, which may approve and which is added on escalation; the approver's role must hold `may_approve` for the action. Visibility is cumulative: Operations sees the least, Finance sees everything Operations sees plus the economics, Management sees everything.

Role	Does	Sees	Never sees
Operations	Quotes, screens wallets, verifies collections (cash after the partner's count), instructs conversions, prepares settlements, requests reroutes, records destination approvals, enters partner rates as quoted	Operational fields, sender-facing prices (including the customer-facing figures the quote prints), partner rates as entered, deal states	Reference rate at capture, spread, margin, pool, cost breakdown, ledger balances, the four variances, FX_RESULT, loss shares, offsets, earnings, reseller margins — on any screen, export, message or report (ROLE.sees_economics = false)

Role	Does	Sees	Never sees
Finance	Approves what Operations initiates (quotes, extensions, rates, conversions, settlements, exceptions); resolves variance exceptions; recommends covered fees and fee caps; records a partner's hold, freeze or decline and its clearance (a screening that is not clear is Management's second look); requests balance conversions, dues recovery, earnings offsets and loss splits; owns reconciliation, the month-end true-ups and ledger adjustments; proposes configuration	Everything Operations sees plus all economics, postings and balances	—
Management	Approves escalations and the heavier actions listed in Section 5 (loss splits, offsets, balance conversions, dues recovery, covered fees and raised caps, own-wallet transfers, post-money voids, own desks); approves configuration, system settings and user changes; countersigns releases when only one Finance user is on shift	Everything	—

2 · The deal: Quote → Collect → Convert → Settle

A deal is a group of one or more legs. A quote-first leg walks the whole top row of Figure 1; a collect-first leg is born at Collected because the money arrived before any quote; an estimate-first leg enters Collecting with an indicative figure and is priced at conversion; a return leg is born at Collected, funded by the returned payout and attributed to its parent leg's group. Pricing fields are stamped once and never recomputed. Fee parts are percentages of the principal at the basis rate and never compound; a target-out quote applies the fee as a markup on the target.

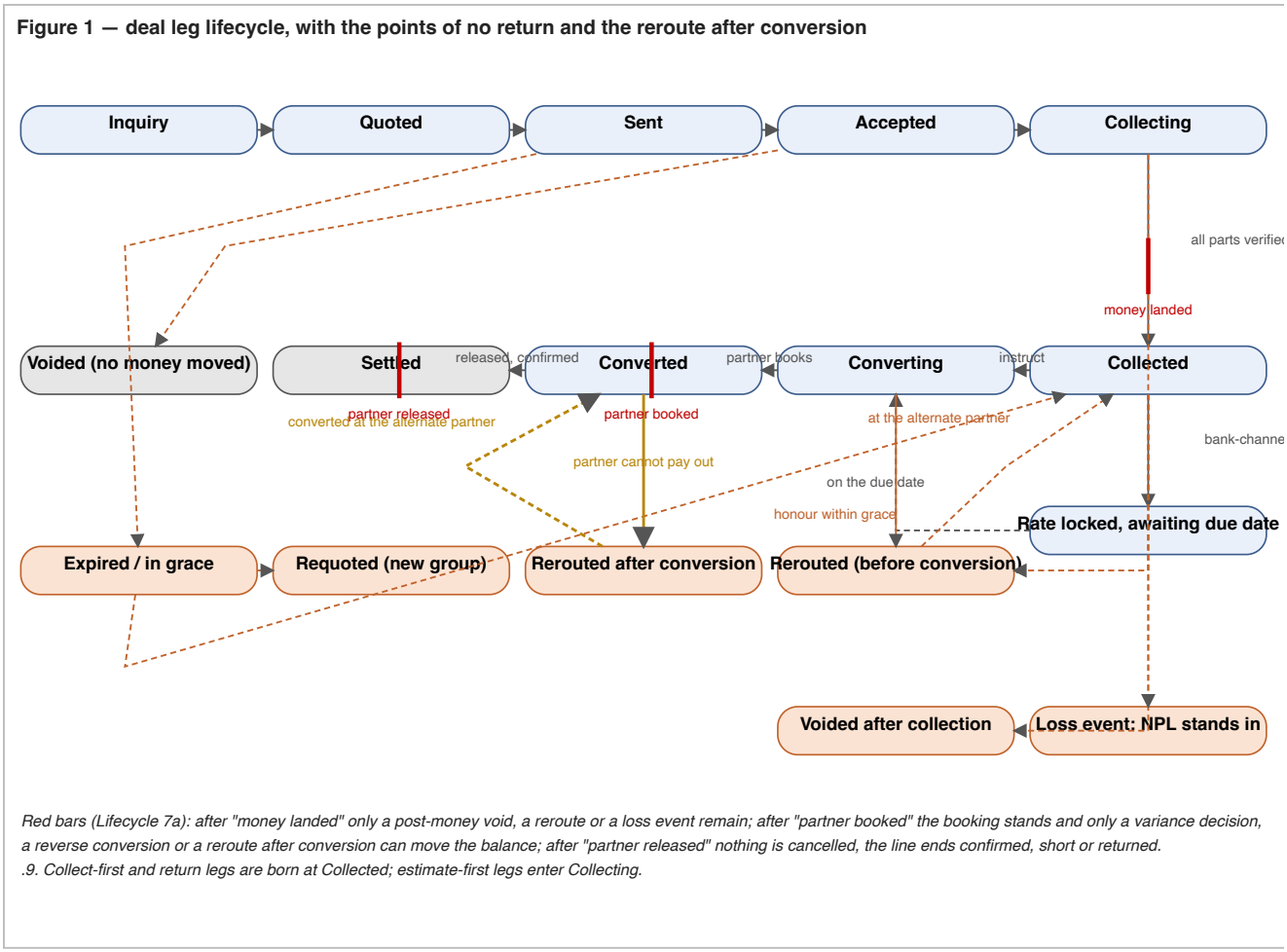


Figure 1 — Deal leg lifecycle. Blue: normal path. Orange: exceptions. Rounded: end states. Red bars: the points of no return (Section 7a). Collect-first and return legs enter at Collected; estimate-first enters at Collecting; a reroute after conversion (Section 2b) returns the leg to Converted at the alternate partner.

Transitions

From	To	Event	Guard	Who
Inquiry	Quoted	Price computed: partner × pair chosen, fee structure and tier applied, rate snapshot taken, day's rate comparison created if missing; the customer-facing fee and rate derived	Partner rate version usable (not breached); margin ≥ floor, else Finance approval; an approved destination exists for the entitled group with that partner; the method is one the partner offers	Ops; Finance if below margin
Quoted	Sent	Quote package issued and marked sent (QUOTE_PACKAGE, template sender-facing only); on a project with Email OTP on, the issue is the approval email: the rendered quote, a one-time code and the partial destination, sent by the email service to the sender's registered approver address, one email per quote issue (2e); the group is then *pending customer approval*	Within quote_freshness_min of the snapshot; otherwise re-price; with Email OTP on, also an approved approver address and chat group on the sender, with no change to them pending	Ops

From	To	Event	Guard	Who
Sent	Accepted	Sender accepts; with Email OTP on, acceptance is the code: the customer pastes it (and, at or above the step-up amount, the amount) in the registered chat group, Operations enters it, the server validates it (right code, right issue, unexpired, unused, amount if required) and stamps the approval with the entering user and the time; the chat screenshot is attached as evidence (2e)	Within the validity in force (effective_valid_until); with Email OTP on, within the code window (default 24 hours): inside validity at the quoted price; after validity only if the drift test passes (grace_drift_pct), otherwise a new issue	Ops records (enters the code)
Sent / Accepted	Sent / Accepted (validity extended)	QUOTE_VALIDITY_EXTENSION: the package re-issued at the same price with a later validity; with Email OTP on, the re-issue is a new quote issue with a new code, and the old code is void	Before the validity in force ends; any number of times; Finance approves, Management added when the new validity passes the partner's cutoff or the rate version's expiry	Ops requests; Finance
Sent / Accepted	Expired / in grace	Validity lapses	Grace applies for grace_min only if market drift ≤ grace_drift_pct; with Email OTP on, the grace for the acceptance is the code window, with the same drift test	System
Sent	Sent (new issue)	Email OTP: any change to the order (amount, rate, fee, method, destination) or a resend supersedes the issue; a new email and a new code go out; the old code is void	Before acceptance	Ops
Sent	Requoted (new deal group) or Voided	Email OTP: the customer rejects, by a reply to the email from the registered address or in the registered chat group; the reason is recorded verbatim	—	Ops records; reason mandatory
Sent	Expired	Email OTP: the code window passes with the code unused; Operations is alerted; a new issue needs a new send	—	System
Expired / in grace	Requoted	Late decision: re-quote, a new deal group; at the sender's request the new leg may be priced on the expired leg's rate by an approved override (honour_expired_rate), the difference against the day's rate posting to VAR_RATE_HONOUR	New deal group reference; the original stamp is kept; Finance approves the override	Ops; Finance
Expired / in grace	Collected	Late decision: honour the old quote within grace	Project late_collection_policy allows it; recorded Finance approval	Finance
Accepted	Collecting	Instructions issued: an active endpoint of the leg's partner entity whose kind matches the collection method, a bridge wallet where the partner lacks the network, or cash instructions with a token	Endpoint active (a bank endpoint only with rail) and of the right kind; a crypto sender's wallet screened unless the project skips up-front screening	Ops

From	To	Event	Guard	Who
Collecting	Collected	All parts verified (hash / slip / receipt; a cash part after the partner's count)	Collected amount matches; each part landed in an active endpoint of the leg's partner entity matching collection_method; a single-use endpoint serves this deal only; overpayment → sender credit; underpayment → wait or void	Ops verifies
Collected	Rate locked, awaiting due date	On a pair that relies on the banking channel the partner confirms the rate after its cutoff	needs_bank_lock; conversion_due_date = the next business day (weekends and HOLIDAY rows skipped)	System
Collected / Rate locked	Converting	Conversion instructed to the partner, optionally keeping the deal's margin unconverted (residual_in)	Rate version still usable; not before conversion_due_date; a next-day rate that breaches the minimum margin defers the conversion with a reason	Ops; Finance approves
Converting	Converted	Partner books; actual_out recorded; margin recognised; the pair's rounding applied; markup-share and rebate accruals posted with the conversion	Variance within tolerance, else an exception opens (the deal does not wait); the difference posts to the account of its cause	System; Finance on exception
Converted	Settled	Client net released by the partner, the promise discharged on release, every line confirmed (full or short) or returned	Settlement lifecycle, Figure 2	—
Collected	Loss event	Money lost after verification: a frozen single-use account, cash found short, a wrong-chain transfer	Management approves the split; the whole loss is booked to NPL and the client made whole; the part stays verified and the deal continues	Ops records; Finance; Management
Inquiry … Collecting	Voided	Void before money moved	Reason recorded; no postings	Ops; Finance
Collected / Converted	Voided after collection	Void after money moved	Disposition: refund (network-fee policy), sender credit, or hold; a refund while the leg is on hold only on the partner's written instruction, with its reference, and Management's approval (2c)	Ops; Management
Converting / Collected	Rerouted → Collected at the alternate partner	Partner unavailable, limit reached, holiday, compliance hold	Approved REROUTE; alternate partner in the applicable set; the money itself not under a partner hold (2c)	Ops; Finance

From	To	Event	Guard	Who
Collecting … Converted	On hold (the leg keeps its state)	A partner reports a hold, freeze or decline on the leg's money or its destination (EMERGENCY_ACTION destination_freeze, REROUTE trigger compliance_hold, LOSS_EVENT account_frozen), or NPL's screening of a wallet on the leg is not clear under the provider's Strict rulebook (second look pending) or stands at reject	The hold is recorded on the leg with its kind (partner_hold / screening_hold) and the event or screening row it comes from; every money-moving action on the leg is refused while it stands (2c)	System on the event; Ops records
On hold	The state it left	Partner hold: the partner's clearance, recorded with its reference. Screening hold: Management's second-look decision accept or accept_alternate, or a new screening row that is clear	Clearance reference present (partner); APPROVAL by Management, approver not the person who screened (screening); the lift is written to AUDIT_LOG with the reference	Finance records; Management decides
On hold	Voided after collection → Refunded, or Rerouted	Disposition under hold: the partner instructs a refund in writing, or the money is moved to a partner the hold does not touch	The partner's instruction reference recorded; Management approves; a refund goes only to the sender's return receiver; never while a partner hold is on the balance itself	Management
Converted	Rerouted after conversion → Converted at the alternate partner	The converting partner cannot pay out (suspended, limit reached, holiday, bank down) and NPL chooses not to wait: the balance is converted back, held in NPL's wallet, forwarded and converted again	Approved REROUTE with phase post_conversion; alternate partner in the applicable set; Management approval; custody within custody_max_hours; the client net stays as priced; any payout instruction already given to the first partner is acknowledged by it as rejected or not executed, with its reference (2d)	Ops; Finance; Management

2a · Effective-date rules

Pricing stamps are write-once, and configuration is versioned by effective date (Configuration Schema, section 1). The four rules below say which version a deal uses; they are the same rules as in the Configuration Schema.

Rule	What happens
1. New versions apply to new pricing only	An approved configuration change takes effect for deals priced on or after its effective date. A deal priced before that date is not re-priced.
2. A priced deal keeps its versions	The fee structure version, the tier, the rate basis, the rounding options, the partner configuration and the policy versions stamped at pricing stay with the deal to the end of its life, including through settlement, return and loss handling.
3. A re-quote uses the configuration of the new pricing time	An expired or requoted leg is priced afresh on the versions in force when the new quote is made (a new deal group); the original leg keeps its stamp.

Rule	What happens
4. A collect-first deal uses the configuration in force when it is priced	A collect-first leg is priced at conversion, so it uses the versions effective at that time, not at the time the money arrived, unless a documented client agreement names a different rule for that project.

2b · Emergencies in the middle of a deal: the three cases

When a partner must be suspended, a destination frozen or a cutoff overridden (the emergency actions of Section 5), the deals already in flight are in one of three positions. The table says what can be done in each.

Case	What can be done	Who	What posts
1. Collection started, not completed (Accepted or Collecting)	Void before money moves: the leg is voided, the instructions withdrawn, a single-use endpoint released. If the sender has already paid part, the deal is in case 2 for that part.	Operations starts, Finance approves	Nothing; no money has moved.
2. Collected, not converted (Collected or Rate locked)	Reroute with phase post_collection: the unconverted money goes through NPL's own wallet to an alternate partner in the applicable set, or is refunded to the sender (a post-money void).	Operations requests; Finance approves the reroute; Management approves a refund	Existing Pattern D of the Ledger Posting Design (T12, T13): HELD, then COLLECTED at the alternate partner.
3. Converted, sitting at the partner (Converted)	Reroute after conversion, phase post_conversion (NPL decision, 20 October): the converting partner converts the client's balance back to the collection currency at its rate of the day, pays it to NPL's own wallet, NPL forwards it to the alternate partner, and that partner converts at its own rate of the day. The client's entitlement is unchanged; the rate difference and the second partner's cost are NPL's.	Operations requests; Finance approves; Management approves, because NPL's wallet holds client money in the meantime	New Pattern O of the Ledger Posting Design (T56 to T58): a balance conversion back, custody, the forward, the second conversion with VAR_CONVERSION and the pool.

Reroute after conversion (phase post_conversion)

Figure 4 gains one entry point. A REROUTE with phase post_conversion starts from a leg in Converted, not from Collected: Requested (Operations, trigger recorded) → Approved (Finance, then Management) → Reconverted (the converting partner has converted the balance back; a BALANCE_CONVERSION owned by the reroute, no markup) → In custody (own wallet) → Forwarded to alternate partner → Completed when the alternate partner has converted and the leg is back in Converted there. The leg's client net is write-once and is what the alternate partner must deliver into the client's balance; what it actually delivers, less its stated cost, differs from that by the rate difference, which is posted to VAR_CONVERSION against NPL's pool at the alternate partner. NPL's margin captured at the first conversion stays where it was earned. Custody is bounded by custody_max_hours and shown on the custody view like any reroute. The first partner's transfer to NPL's wallet and NPL's forward to the alternate partner are both outbound transfers under the controls of Section 5. A reroute after conversion is also an alternative payout, so Section 2d applies: it starts only when any payout instruction already given to the first partner is acknowledged by that partner as rejected or not executed.

2c · Holds: scope, allowed actions, release and audit

A hold is a named condition on a deal leg, not a narrative. It comes from one of two sources and keeps the leg in the state it was in: a **partner hold** (the partner, or its bank, reports a hold, freeze or decline on the client's money or on a destination: NPL records it as the event it is and does not decide it) or a **screening hold** (NPL's own screening of a wallet on the leg is not clear under the provider's Strict rulebook and awaits Management's second look, or a decision of reject stands). NPL has no compliance function; the partner's decisions are the partner's, and the only decision NPL takes is Management's second look on its own screening (Roles & Visibility rule 11).

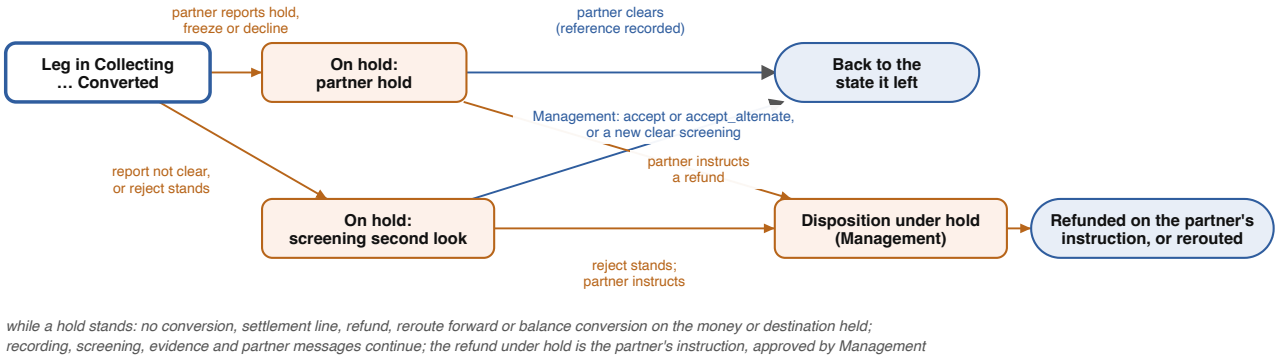


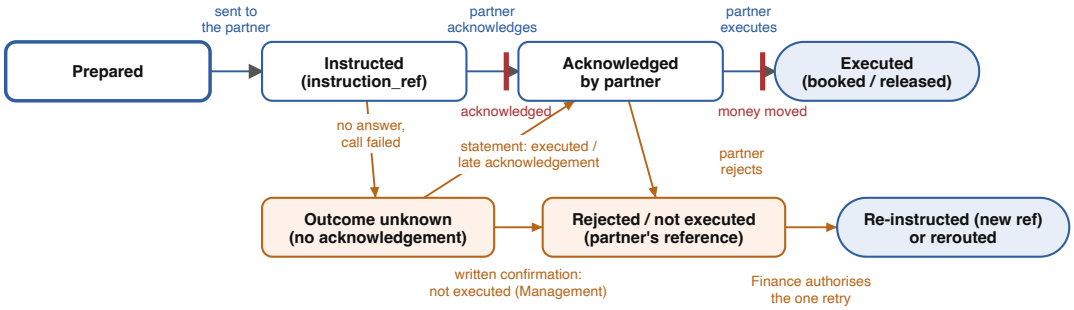
Figure 1a — A hold on a deal leg. The leg keeps its state; a partner hold ends only on the partner's clearance, a screening hold only on Management's second look or a new clear screening; the one movement under a hold is the disposition the partner instructs, approved by Management.

Question	Rule
Scope	The hold attaches to the leg and names what it covers: the collected money (partner hold on the balance, or a screening hold on the sender's wallet), or the destination (partner freeze of the destination, or a screening hold on the receiver's wallet). A partner hold on a balance covers every leg whose money rests in it.
Blocked while it stands	Every action that moves the held money or pays the held destination: instructing a conversion or a balance conversion, preparing or releasing a settlement line, a refund or sender credit paid out, a reroute forward from custody, a return leg. The server refuses them with the hold as the reason, whichever role asks and whichever channel (screen, API, a replayed request).
Allowed while it stands	Recording and evidence: a new screening row, partner messages and their references, the exception list, verification of a part that has already landed, a void before money moved on a leg that has not collected, and settlement lines of other legs that neither draw on the held money nor pay the held destination.
Who lifts it	A partner hold: nobody at NPL; it ends when the partner's clearance is recorded with its reference (Finance records it; it is an audited record, not an approval). A screening hold: Management's second-look decision accept or accept_alternate (an APPROVAL under screening_second_look, approver never the person who screened), or a new screening row that is clear. Finance has no right over either.
Disposition	While a hold stands, the only money movement is the one the partner instructs in writing (a refund to the sender's return receiver, or a transfer the partner directs); it is recorded with the partner's reference and approved by Management, and never happens while a partner hold is on the balance itself. The reviewer's case, funds from a newly flagged wallet and a partner asking for a refund, is exactly this path: Operations cannot issue the return transfer; Management approves it on the partner's instruction.
Expiry and re-screening	A hold does not expire by time. A screening hold is re-examined only by a new screening row (new report, new decision); a partner hold is chased daily from the exception list with its age (Section 8). A leg on hold for longer than the project's limit is escalated to Management by the time-limit rule of 8a.
Audit	The start and the end of every hold are AUDIT_LOG rows naming the event or screening row, the reference, the actor and the time; the second-look decision is an APPROVAL; every refused action under a hold is logged with the hold as its reason. The ERD gains the hold on the leg (hold_kind, hold_ref) and the tests of the Test Strategy (S-HOLD) attempt a payout, a conversion, a refund and a reroute under each kind of hold.

2d · External instructions: prepared, acknowledged, executed

Every instruction NPL gives a partner — a conversion, a balance conversion, a settlement line, the forward of a reroute — is one row with its own reference (instruction_ref, the idempotency key of the Architecture) and an acknowledgement status that distinguishes three facts the reviewer asks to keep apart: the instruction is **prepared** at NPL, the partner has **acknowledged** it (from then on it cannot be withdrawn: the first point of no return), and the partner has **executed** it (the money moved: the second). A line is "sent" only on the partner's acknowledgement; an instruction with no acknowledgement is "outcome unknown" and stays so until the partner's statement or confirmation says executed or not executed. The two held states are different things: "sent,

not yet confirmed" (Ledger Posting Design rule 13a) is an acknowledged instruction waiting for its execution to be confirmed, and posts nothing; "sent, outcome unknown" is an instruction the partner has not acknowledged at all. A line enters "outcome unknown" when the partner's acknowledgement has not arrived by the acknowledgement time-out of the instruction (the partner's configured wait after the call; a failed call counts the same), and Operations raises a partner reference inquiry the same day, recording the partner's answer with its evidence on the instruction.



outcome unknown: on the exception list with its age; Finance obtains the partner's statement; a Management exception needs the partner's written confirmation and the reconciliation (2d)

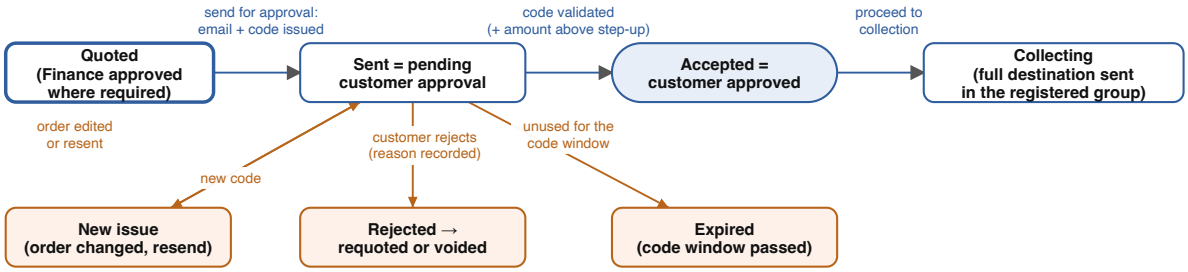
Figure 2a — An external instruction (conversion, settlement line, reroute forward). Two points of no return: the partner's acknowledgement (no withdrawal) and the execution (money moved). One open instruction per promise: a second one is refused while the first is instructed, acknowledged or unknown.

1. **One open instruction per promise.** While an instruction for a promise (a settlement line, a conversion of a balance) is instructed, acknowledged or of unknown outcome, the server refuses a second instruction for the same promise from any role and any channel: the unique key is the promise, not the request (Ledger Posting Design rule 1b, Pattern P). This is what prevents a double payment.
2. **A reroute or any alternative payout starts from a closed instruction.** A reroute after conversion, a re-instructed settlement line or a payout through another partner is allowed only when the original instruction's status is rejected, or acknowledged as not executed, with the partner's reference recorded on the instruction, and the balance reconciliation shows the client's money still at the first partner (ledger balance equal to the partner's statement, the amount included).
3. **Unknown is not rejected.** An instruction with outcome unknown is on the exception list with its age; Finance obtains the partner's statement or written confirmation; nothing is re-instructed meanwhile. Management may authorise an exception only with the partner's written confirmation that the instruction was not and will not be executed, its reference, and the reconciliation attached; the exception is an APPROVAL naming them, and the instruction is closed as rejected before the alternative is instructed. Three ways out, each recorded on the held instruction: **executed** — the partner's statement or reference shows the money moved; the instruction is marked acknowledged and executed against that reference and the line continues as usual, nothing posted twice; **not received** — the partner confirms in writing that it never received the instruction and will not execute it; Management closes the instruction as rejected on that confirmation (the exception above) and Finance authorises the one retry under a new partner reference, recorded as a retry of the held instruction (retry_of), never a second open instruction; **answered late** — the acknowledgement arrives after the time-out; it is recorded as the acknowledgement with its real time and the line proceeds as acknowledged. The Test Strategy test G-EXT is this rule at acceptance level.
4. **Delayed confirmation is not failure.** The reviewer's case, the first partner debits EUR but confirmation is delayed, is an acknowledged instruction: the alternative payout is refused until the partner's statement arrives, and when it shows the money moved the line ends confirmed as usual. The Test Strategy test S-ACK exercises every acknowledgement outcome (acknowledged late, rejected, unknown then executed, unknown then not executed) and proves no second instruction can be created while the first is open.

2e · Email OTP: the customer's approval of the order

Today a customer's consent to an order is a message in a chat group that the system never sees. **Email OTP** is an optional feature, switched on per project, that makes that consent a recorded step of the deal: after the quote is

prepared (and Finance has approved it where the thresholds require), the customer's registered approver receives one email with the order as quoted and a one-time code, pastes the code in that sender's registered chat group (each sender has its own group and its own approver address), and Operations enters it. The code is the acceptance. In the states of Figure 1 nothing new is needed: *Sent* is the quote awaiting the customer's code, *Accepted* is the code validated; the screens label them *pending customer approval* and *customer approved*, and the approval row carries its own stamps. Projects with the feature off follow today's flow unchanged. The design comes from the reviewer's change request CR-01 of October 2026 and is a priced change to the build scope.



the email: the rendered quote, a one-time code, the partial destination (first and last characters); no link, never the full account or wallet; one email per issue;
the approval: the code pasted in the registered chat group, entered by Operations, validated by the server; the chat screenshot is the evidence

Figure 1b — Email OTP: the customer's approval of a quote (projects with the feature on). Sent is 'pending customer approval'; Accepted is 'customer approved'. The code lives for the code window; the price for the quote's validity; between the two, the drift test decides.

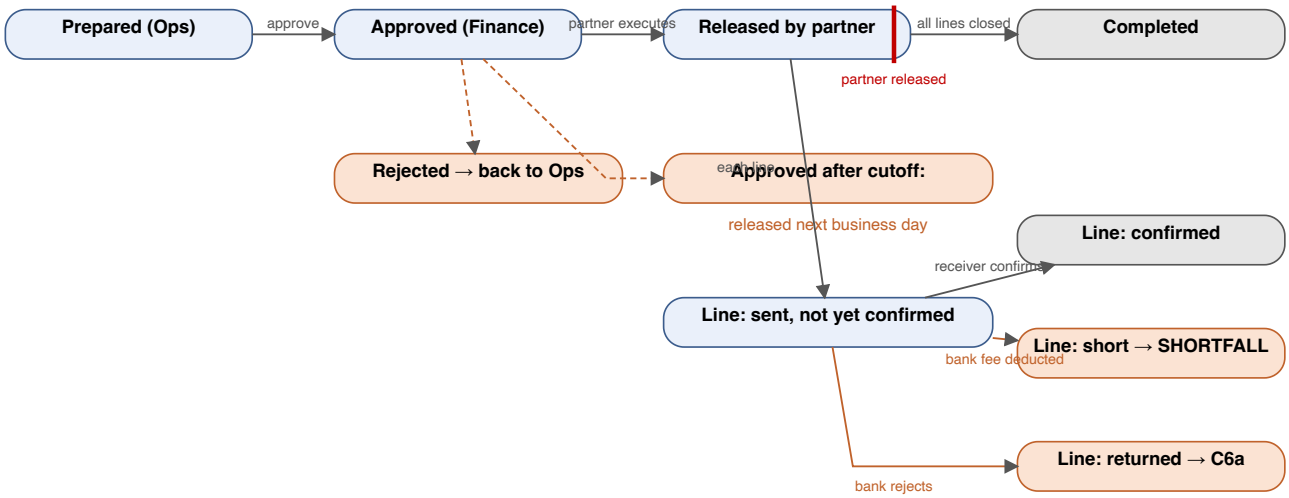
Question	Rule
What the email carries	The quote issue as rendered (deal reference with the issue number, client, pair, amount in, the sender-facing rate at five decimals, fees, amount out, collection method, validity), a one-time code, its expiry in the project's time zone, and, where the collection destination exists at that point, a **partial destination**: the first and last four characters of the collection wallet, or the last four digits of the receiving account; nothing for cash collection, where the token at handover is the control. The instruction printed: paste the code in the registered chat group to approve; reply with 'reject' and the reason to reject; the full destination will follow in the registered group and must match the partial above; NPL never creates new groups.
What it never carries	A link or button that approves or rejects (mail scanners open links, and a link invites a swapped destination), the full wallet or bank account, or anything of the hidden field groups. The email is information and a code; the approval happens across two channels the customer already has, the registered inbox and the registered chat group. A cloned chat group cannot produce the email leg, cannot use a code (codes approve only the real issue, entered by Operations from the registered group) and cannot substitute a destination that survives the partial-identifier check.
The code	Eight or more characters from an unambiguous alphabet, generated at random, stored only as a hash, bound to the deal group, the quote issue and the approver address it was sent to, valid for the project's code window (default 24 hours), consumed on its first successful validation. Five failed attempts lock it; a locked, expired or voided code needs a new send. A code is never shown on any screen or report.
Step-up for large orders	At or above the project's step-up amount (default USD 50,000 equivalent), the customer's paste must carry the code and the order amount, and Operations enters both; a code alone does not validate.
One issue, one code	The code approves exactly one quote issue. Any change to the order after sending (amount, rate, fees, method, destination) and any resend is a new issue: a new email, a new code, the old code void. The emailed snapshot is stored with the deal, content-addressed like a bank slip. Exactly one email per issue.

Question	Rule
The two clocks	The quote's validity decides the price; the code window decides how long the customer has. Inside validity the approval counts at the quoted price. After validity but inside the code window it counts only if the rate has moved less than the project's drift tolerance at that moment (grace_drift_pct, the existing rule); otherwise a new issue. After the code window the code is expired, the group is back at Quoted and Operations is alerted. The screens show a countdown on the work list and raise three alerts: a code nearing expiry, a rejection received, and a change to a sender's approver address or chat group pending Finance's approval.
Tripwire	If a code is issued and nothing from the registered chat group has been recorded against it within the tripwire window (default 4 hours), an alert tells Operations to verify with the customer through a known contact outside the chat. A validated code whose chat evidence cannot be produced from the registered group blocks collection and opens an exception.
Rejection	A reply to the email from the registered approver address, or a message in the registered chat group, is recorded as a rejection with its reason verbatim; the group leaves Sent (re-quoted as a new group, or voided). Nothing that arrives by email ever approves; a reply from any other address is recorded and ignored.
Evidence and audit	The approval record carries the stored email snapshot, the code reference, the validation time, the entering user and the chat screenshot; a rejection carries its reason. Every send, resend, delivery failure, bounce, reply, validation attempt, lock, void and tripwire is an event on the approval and an AUDIT_LOG row.
The settings, protected	The project holds the switch and the three windows; each sender holds exactly one approver email and one registered chat group (platform, group name, group id), because every sender has its own chat group with NPL. While the switch is on, a quote for a sender without an approved address and group cannot be sent, and the screen says why; there is no fallback. Operations enters, Finance approves, never the same person; a new approver address activates only after a verification code sent to it comes back through that sender's chat group; while a change is pending no approval email can be sent for that sender; a change of address voids that sender's open codes only, so a live code never moves; the sender row shows the current address, the audit log the history. Switching the feature off is the same controlled change; groups already pending complete or expire under the rules in force when sent (Roles & Visibility rule 13; Configuration Schema rule 16).

Operations sends the full deposit wallet or account only after the approval, in the registered chat group, and the customer's standing rule is to pay only if it matches the partial identifier in the email. The acceptance test is S-OTP (Test Strategy).

3 · Settlement, rates, reroute and approval records

Figure 2 — settlement: a run and its lines, with the named "sent, not yet confirmed" state



Sent, not yet confirmed (NPL confirmed 20 October): the promise is already discharged; the line shows released_at and expected_confirmation_by and is on the Finance log report and, past its expected time, on the exception list. Management approves above the settlement limit, a third same-day slot, a cross-group payout, or as countersign.

Figure 2 — Settlement: a run and its lines. The promise is discharged when the partner releases the money; a line is then "sent, not yet confirmed" until the receiver confirms, the bank deducts a fee or the bank rejects; a return or a short delivery re-raises the promise. A run completes when every line is confirmed, short or returned. Red bar: the point of no return.

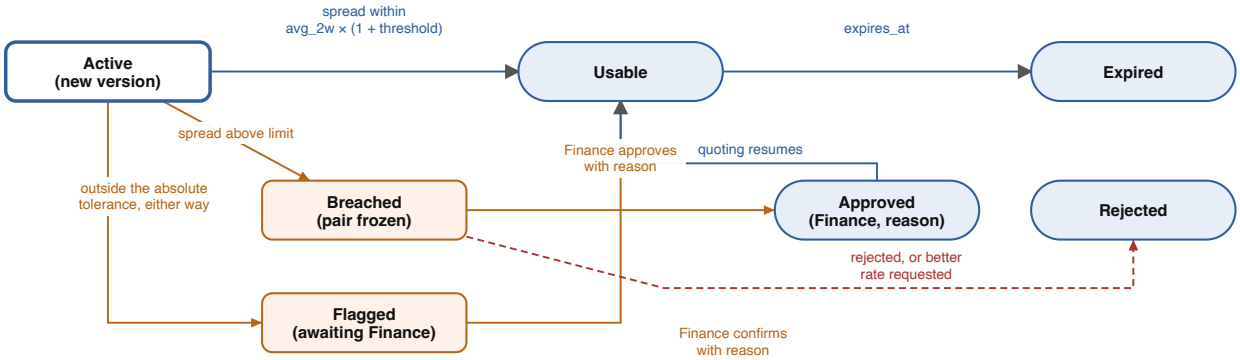
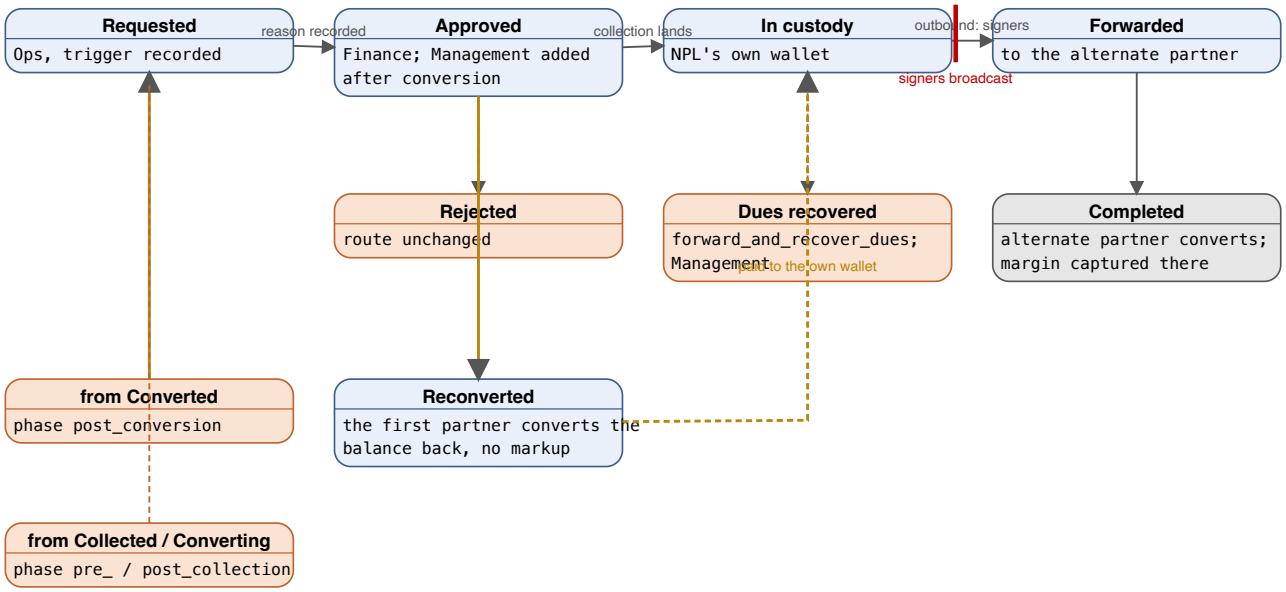


Figure 3 — Partner rate version. Stored as quoted, with its direction and, for cash pairs, its city. A breach freezes quoting and conversion on that partner × pair; there is no skip path.

Figure 4 — reroute, with the entry after conversion



Client money rests in NPL's own wallet in four cases now: before conversion, after conversion, a bridge collection, an own-desk leg.
The leg's client net is write-once; the alternate partner's cost and the rate difference are NPL's (Ledger Pattern O).

Figure 4 — Reroute, with the entry after conversion (phase post_conversion, Section 2b). Client money rests in NPL's own wallet in four cases only — an approved reroute before conversion, a reroute after conversion, a bridge collection awaiting its hop, an own-desk leg — and every resting balance carries deal, reason and age. Red bar: the point of no return for an own-wallet transfer. A reroute after conversion starts only from a closed external instruction with the partner's reference and a reconciled balance (Section 2d, Figure 2a).

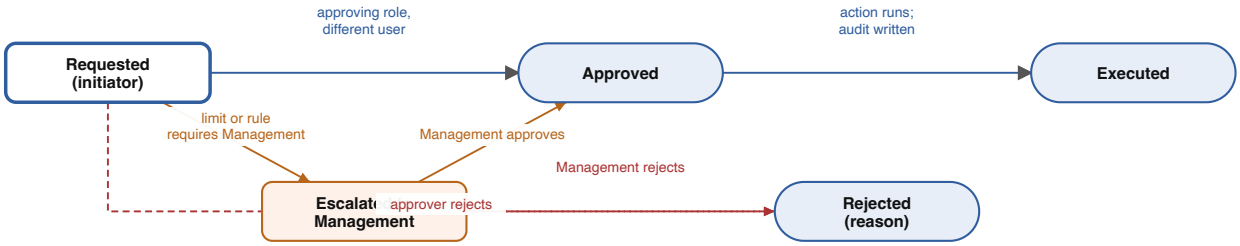


Figure 5 — An APPROVAL record. The approver's ROLE must hold may_approve for the action in ROLE_RIGHT and differ from the initiator; a rejection never deletes anything.

4 · Losses, offsets and the reseller's invoices

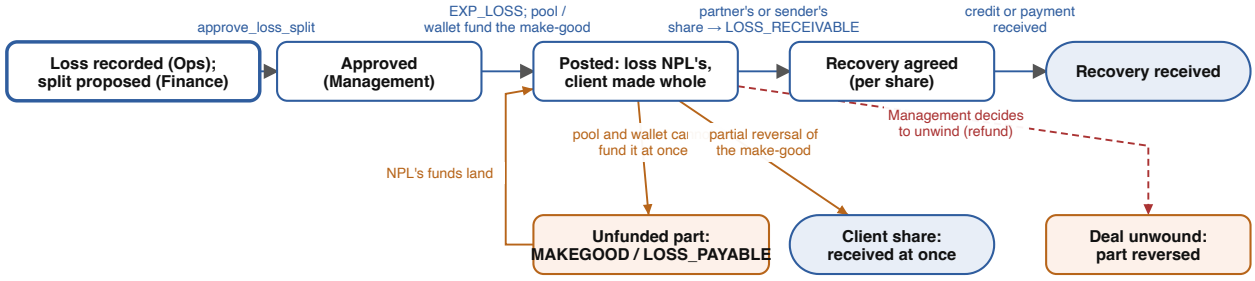


Figure 6 — Loss event and recoveries. The whole loss is NPL's first and the client is made whole in the same transaction; each share is then its own LOSS_RECOVERY posting. The collection part stays verified unless Management unwinds the deal.

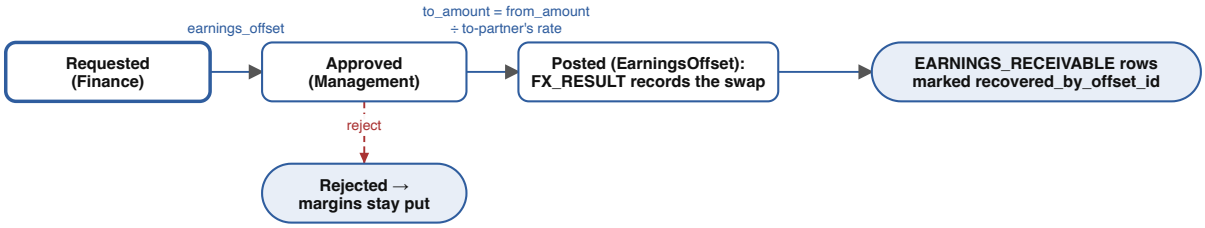


Figure 7 — Earnings offset between partners. NPL's converted margin at one partner becomes the client's there; the same value of the client's unconverted money at another becomes NPL's, at the to-partner's rate of the day; the client's entitlement is unchanged.

NPL-GR's two invoices

In the reseller project NPL-GR is the client of its own remittance deal. The vendor's invoice to NPL-GR and NPL-GR's resale invoice to its customer are posted once each, in the invoice currency, when received and when issued; the resale invoice names the vendor invoice it marks up. A deal group that collects the resale invoice names it and the part it collects (advance, then balance) at the pricing rate; each vendor payment line names the vendor invoice, the obligation it discharges and the rate at discharge, and the forex result of paying on a later day's rate is a true-up on the remittance side, never on the reseller revenue. A resale invoice is settled when its groups have collected its amount; a vendor invoice when its released lines have discharged its amount (INVOICE_BALANCE by kind).

5 · Controlled actions: who initiates, who approves, what rejection means

The approving role is always a different user from the initiator. "Escalates to Management when" lists the conditions that add a Management approval on top of, not instead of, the approving role's. The last column is the `ROLE_RIGHT.action` code that carries the right; "—" marks an audited operational step with no approval row of its own.

Controlled action	Initiates	Approves	Escalates to Management when	On rejection	ROLE_RIGHT
Quote a deal (price, fee, partner × pair)	Operations	Finance	Below-margin quote: Finance approval mandatory before sending (FEE_STRUCTURE.min_margin_pct)	Returns to Ops with reason; the quote package is not sent; Ops re-prices as a new quote	`price_quote`
Extend a quote's validity (before the validity in force ends)	Operations	Finance	The new validity passes the partner's cutoff or the rate version's expiry; any number of extensions, each a QUOTE_VALIDITY_EXTENSION row re-issuing the package at the same price	Validity ends as quoted; the paths are the grace decision and a new deal group	`extend_quote_validity`
Honour an expired quote's rate on a new deal (late decision)	Operations	Finance	Always Finance (honour_expired_rate): the new leg is priced on the earlier leg's market rate and partner rate version, the difference against the day's rate posting to VAR_RATE_HONOUR	Re-quote at today's rate with a new deal reference; the original stamp is kept	`honour_expired_rate`
Enter a manual rate (reference feed outage)	Operations	Finance	Evidence attached (both feeds down)	No rate; quoting waits for the feed or a new manual entry	—
Confirm a partner rate version	Operations (enters, as quoted: rate and direction,; city for cash pairs)	Finance (confirms)	Breached version: Finance approval with recorded reason, or rejection; no skip path	Pair stays frozen; Ops may request a better rate from the partner, which is a new version	—
Record a partner cutoff override (extension agreed, or early close)	Finance	— (recorded with the partner's agreement reference)	—	The standing cutoff applies	—

Controlled action	Initiates	Approves	Escalates to Management when	On rejection	ROLE_RIGHT
Send a quote for the customer's approval (Email OTP)	Operations (only on a project with Email OTP on, after any Finance approval the quote needed)	— (no approval: the send is the issue of the quote)	One email per quote issue to the sender's registered approver address: the rendered quote, a one-time code and the partial destination; no link, never the full account or wallet; a resend or any change to the order is a new issue and voids the old code; refused while the sender has no approved approver address and chat group, or a change to them is pending	—	`send_email_otp`
Enter the customer's approval code (Email OTP)	Operations (from the customer's paste in the registered chat group)	— (the server validates: right code, right issue, unexpired, unused; the amount too above the step-up threshold)	Five wrong attempts lock the code; a locked or expired code needs a new send; the chat screenshot is attached as evidence; the approval stamps the group Accepted with the entering user and the time; a code whose evidence is not from the registered group blocks collection and opens an exception	A wrong code is refused with the attempt counted	`enter_email_otp_code`
Record a customer rejection (Email OTP)	Operations (a reply from the registered approver address, or a message in the registered chat group)	— (reason mandatory, recorded verbatim)	The group leaves Sent: re-quoted as a new group or voided; the code is void; a rejection never needs an approval	—	`record_customer_rejection`
Change the Email OTP settings (the project's switch; a sender's approver address and registered chat group)	Operations	Finance (never the entering user)	Protected master data (Roles rule 13): a new approver address activates only after a verification code sent to it comes back through the registered chat group; a chat-group change carries a screenshot from the group; while a change is pending no approval email can be sent for that sender; a change of address voids that sender's open codes	The pending change is dropped with the reason; the old values stay in force	`set_email_otp_settings`

Controlled action	Initiates	Approves	Escalates to Management when	On rejection	ROLE_RIGHT
Screen a sending or receiving wallet	Operations (an Operations action, not an approval)	— (result and decision recorded with evidence); Management decides a second look (screening_second_look)	A report not clear under the provider's Strict rulebook (any category at threshold, a terrorism / TF trace, or a score outside the clear band) goes to Management's second look: Management decides accept, accept_alternate or reject with a reason; Finance has no right on a screening; a partner's hold, freeze or decline is recorded as its event and never lifted at NPL; accept_alternate may land only on an elevated-band endpoint	No collection on that address; the sender is given an alternate wallet or refused	`screening_second_look`
Verify a collection (hash / slip / receipt; a cash part after the partner's count)	Operations	— (audited; Finance reviews in reconciliation)	Underpayment left open beyond policy: Finance decides wait or void. The system refuses the part if its endpoint is not active, does not belong to the leg's partner entity, or its kind does not match the leg's collection method; a single-use endpoint serves one deal only	Collection stays unverified; funds are not attributed	—
Activate a collection endpoint (COLLECTION_RECEIVING_ENDPOINT)	Operations	Finance	— (the system refuses to activate a bank endpoint without rail, or a wallet whose rail differs from its network —; a wallet's network must be one the partner's vehicle offers)	Endpoint stays inactive; no collection may reference it	—
Instruct a conversion	Operations	Finance	Variance beyond tolerance opens an exception that Finance resolves; on a bank-channel pair Converting may not start before conversion_due_date; a next-day rate that breaches the minimum margin defers the conversion with a reason	Conversion not instructed; deal stays Collected	—
Keep the deal's margin unconverted (CONVERSION.residual_in)	Finance	— (part of the conversion instruction; EARN_GROSS recognised in the collection currency)	—	The margin converts with the client's money and rests at the partner as EARNINGS_RECEIVABLE	—
Resolve a conversion variance exception	Finance	Finance (second user)	Variance above the project's exception limit → Management	Exception stays open; the deal is already converted and does not wait	—

Controlled action	Initiates	Approves	Escalates to Management when	On rejection	ROLE_RIGHT
Prepare and release a settlement	Operations	Finance	Above settlement limit; third same-day slot; a counterparty receiver paid from a leg attributed to another group (offset) — never for a line paying a sender_return or partner_transit receiver, which debits the group of the leg named in its deal_id; a bank-rail settlement approved after the partner's cutoff goes out on the next business day; only one Finance user on shift → Management countersigns	Back to Prepared with reason; lines can be changed and re-submitted	—
Override the bank-fee treatment on one settlement	Operations	Finance	—	Project default applies	—
Cover a bank fee, or raise the fee above the cap	Finance (recommends, FEE_DECISION.recommended_by)	Management	Always: a covered bank fee, or a markup above max_markup_pct after discussing with the customer	The project's bank-fee treatment applies; the cap stands	`cover_bank_fee / raise_fee_cap`
Top up a shortfall from the pool	Finance	Management	Always Management (company money becomes client money)	Shortfall stays open; receiver remains short	—
Pay a counterparty receiver from another group's leg (offset)	Operations	Finance	Always Management, with tighter aging on the resulting offset. Does not apply to refund (sender_return) or hop (partner_transit) lines: cross_group_payout never fires on them	Lines must stay within the entitled group	`cross_group_payout`
Request a reroute	Operations	Finance	Contingency route set (applicable_for_reroute) is configuration: Management	Deal stays on its route; if the partner is unavailable the deal waits or is voided	—
Outbound transfer from NPL's own wallet	Finance	Management (dual approval)	Always; executed by NPL signers outside the platform (hardware or multisig); a bridge hop to the leg's partner within custody_max_hours follows the same path	Funds stay in custody; aging continues to be reported	—
Recover dues on a reroute (net against forwarded funds)	Finance	Management	Always; amount ≤ outstanding receivable at that partner; partner's agreement recorded	Forward the full amount	`recover_dues`

Controlled action	Initiates	Approves	Escalates to Management when	On rejection	ROLE_RIGHT
Earnings offset between partners (EARNINGS_OFFSET)	Finance	Management	Always: NPL's converted margin at one partner becomes the client's there and the same value of the client's unconverted money at another becomes NPL's, at the to-partner's rate of the day; recovers EARNINGS_RECEIVABLE rows; recorded for NPL in FX_RESULT	Margins stay where they rest; receivables wait for the partner's fee cycle	`earnings_offset`
Balance conversion (client balance to another currency)	Finance	Management	Always; any excess over the served group's entitlement is an offset and needs the offset approval too	Balance stays in its currency	`balance_conversion`
Open a return leg after a rejected payout	Operations	Finance	Destination other than the sender's own return receiver → Management with reason (refund_to_third_party); markup apply / waive is a FEE_DECISION approved by Finance	Returned funds stay in balance for later payouts	`refund_to_third_party`
Override the network-fee policy on a refund	Operations	Finance	—	Project default applies	—
Void a deal before money moved	Operations	Finance	—	Deal continues	—
Void a deal after collection (refund / sender credit / hold)	Operations	Management	Always	Deal continues; funds stay attributed	—
Record a loss event and approve its split	Operations records; Finance proposes the split	Management (approve_loss_split)	Always: the whole loss is booked to NPL and the client made whole when the event is approved; each share the partner, the sender or the client bears is its own LOSS_RECOVERY posting; unwinding the deal (COLLECTION reversed) is a separate Management decision	The loss stays open on the exception list; the make-good stands	`approve_loss_split`
Record a loss recovery received (partner credit, sender payment)	Finance	— (audited; names the LOSS_RECOVERY row)	—	The receivable stays open	—
Record a partner's destination approval (DESTINATION_APPROVAL)	Operations	Finance	— (payout details follow the account's kind: bank reference for a bank account, network and address for a wallet —; a wallet account carries its screening)	Approval stays pending_partner; quote-first deals for it cannot leave Inquiry	—
Post NPL-GR's vendor and resale invoices	Finance	— (audited; posted once each in the invoice currency, source Invoice)	—	Nothing posts; the remittance deal cannot name an invoice that is not recorded	—

Controlled action	Initiates	Approves	Escalates to Management when	On rejection	ROLE_RIGHT
Enable an own-desk partner configuration	Finance	Management	Always: NPL itself converts in its own wallet for this project	The project keeps its outside partners	`enable_own_desk`
Change thresholds, fee structures and tiers, partner configuration, rate sources, pairs and rounding, holidays, templates	Finance	Management	Always; all versioned (new rows with an effective date)	Current version stays in force	–
Change a system setting (reporting currency, staleness limit, time zone) or a platform pair	Finance	Management	Always	Current setting stays in force	–
Create, change or remove a user or role (ROLE, ROLE_RIGHT, USER)	Management	Management (second user)	Always	No change	–
Ledger adjustment (reversal and re-posting; a company currency exchange through FX_RESULT)	Finance	Management	Always; the reversal names the transaction it reverses	Books unchanged; the discrepancy stays on the exception list	–
Emergency suspend of a partner, freeze of a destination, or override of a partner cutoff (immediate)	Finance applies it at once, with a mandatory reason; the suspension, freeze or override takes effect immediately on every deal and settlement it touches	Management ratifies within one business day (after the fact)	Always: the action is recorded as applied and awaiting ratification; the deals affected show it as the blocker (Roles & Visibility rule 8)	Not ratified: the action is lifted and the standing configuration applies again; the record and the reason stay (EMERGENCY_ACTION in the ERD)	`emergency_suspend`

Deals in flight when an emergency action is applied are handled by the three cases of the table after Figure 4.

6 · Rejection handling, the same way everywhere

1. **Nothing is deleted.** The request, the rejection and its reason stay on the APPROVAL record and in the audit log. A re-submission is a new approval request that points at the rejected one.
2. **The record returns to the initiator's last safe state.** A rejected quote is not sent; a rejected settlement goes back to Prepared; a rejected reroute leaves the route unchanged; a rejected balance conversion leaves the balance in its currency; a rejected offset leaves every margin where it rests.
3. **Money that already moved is never rolled back by a rejection.** A conversion the partner has booked stands; a variance exception is resolved by a decision and, if needed, a reversing ledger entry, not by editing the conversion. A loss that Management has approved stands as NPL's; only its shares are recovered.
4. **A rejection reason is mandatory** and is visible to the initiator; derived-economics fields are stripped from the reason shown to Operations.
5. **Frozen states need a decision, not time.** A breached rate version, an open variance exception, an open offset, custody in NPL's wallet, an unfunded make-good and a negative pool all stay on the daily exception list until a role with authority acts. A leg on hold (2c) and an instruction with outcome unknown (2d) are on the same list and end the same way: by a recorded decision or reference, never by time.
6. **Escalation never skips a level.** Management approves in addition to Finance; when only one Finance user is on shift, Management's countersignature stands in for the second Finance user.

6a · Concurrency rules

Section 5 of the Architecture note already gives every money-moving action a unique key, so a retried request cannot post twice, and locks the record a person is editing. Three rules complete that protection.

Rule	What the system does
1. An approval is valid only for the state the approver saw	An APPROVAL carries the record version it was requested on. If the record changed between request and approval (another user edited it, a job moved it, a second approval landed), the approval is refused and the approver is shown the current state; a fresh request is needed. Two approvers acting on the same request at once: the first commit wins, the second is refused.
2. Two runs of a job never overlap	Each background job of the Architecture note (Section 4) takes a lock row before it runs and releases it at the end; a run that finds the lock held exits at once and is recorded as skipped. After a restart the job catches up from where the last run finished, so nothing is processed twice.
3. A record lock expires	RECORD_LOCK is released on save or cancel; it expires after a fixed idle time (THRESHOLD kind lock_idle_minutes, default 15) so a closed laptop does not block a record for the day; Management can clear a lock; the lock, its release, its expiry and any clearance are written to AUDIT_LOG. The lock row carries lock_expires_at (ERD Appendix B.5).

These rules are built in Phase 1 (M3 to M5) and tested with the concurrency, duplicate-submission and outage cases of the go-live acceptance tests (Cutover Approach, exit criteria 9 to 13).

7 · State tables by entity

Entity	States and transitions	Notes
DEAL_GROUP	Inquiry → Quoted → Sent → Accepted → Collecting → Collected → Converting → Converted → Settled; Expired / in grace; Requoted; Voided; Voided after collection → Refunded / Credited / Held (with Email OTP on, Sent is shown as pending customer approval and Accepted as customer approved; 2e)	The group's state follows its legs: Settled when every forward leg is settled and no return leg is open. In NPL-GR the group names the resale invoice it collects and the part of it.
DEAL (leg)	As Figure 1. A collect-first or return leg is born at Collected; an estimate-first leg enters Collecting; leg 2 of a route is born at Collected when leg 1 pays the transit receiver. On a bank-channel pair a Collected leg carries rate_locked_at, cutoff_at and conversion_due_date and may not convert before the due date	leg_type and funding_source decide the entry point. Pricing fields are write-once after priced_at; a leg priced on an expired leg's rate carries rate_reused_from_deal_id and the override that allowed it. A return or hop leg inherits the parent leg's receiver group; cross_group_payout never fires on it.
QUOTE_PACKAGE	issued → superseded (by a re-issue)	One row per issue of a group's quote; a quote-first leg is Sent only when a package has sent_at; freshness runs from the leg's market snapshot to sent_at, validity from sent_at. With Email OTP on, the issue is the approval email (channel email) and its code belongs to that issue; a re-issue voids the code.
CUSTOMER_APPROVAL	issued → approved / rejected / expired / voided / locked	One row per code sent (2e): the quote issue, the code reference and hash, the amount expected above the step-up threshold, the partial destination printed, the email snapshot, the attempts, the decision with its stamps and the chat evidence. The decision is write-once; every send, delivery event, reply, attempt, lock and tripwire is a CUSTOMER_APPROVAL_EVENT row.
QUOTE_VALIDITY_EXTENSION	requested → approved (Finance; Management past the cutoff or the rate version's expiry)	Any number per leg before the validity in force ends; each moves DEAL.effective_valid_until and re-issues the package at the same price.
WALLET_SCREENING (decision)	accept | accept_alternate | reject; a report not clear under the provider's Strict rulebook awaits Management's second look, a screening hold on the leg (2c)	An Operations action before a crypto collection or before a wallet payout; accept_alternate lands only on an elevated-band endpoint. Finance has no right on a screening; a reject stands until a new screening row decided by Management (Roles & Visibility rule 11).
DEAL (hold)	none → partner_hold | screening_hold → none (cleared); → disposition (refund on the partner's instruction, or reroute)	The leg keeps its state; hold_kind and hold_ref name the event or screening row; every money-moving action on the leg is refused while the hold stands; lifted by the partner's clearance reference or Management's second look (2c).

Entity	States and transitions	Notes
COLLECTION	pending → verified; reversed (only when Management unwinds a deal after a loss). Cash: token_issued → handed_over → counted → verified	Parts post as verified; a cash part only after the partner's count. Overpayment creates SENDER_CREDIT; underpayment waits or is voided. After a loss the part stays verified: NPL's funds stand in.
CONVERSION	Expected → instructed → acknowledged → Booked → Approved; Variance exception (Booked, beyond tolerance) → Resolved; instructed → outcome unknown → acknowledged | rejected → re-instructed under a new reference, as a retry (2d)	The booked figure is what the partner did; the difference posts to VAR_CONVERSION (partner_calculates), VAR_CUTOFF (after the cutoff) or VAR_RATE_HONOUR (an honoured expired rate); the exception is resolved by Finance; the deal does not wait. residual_in keeps the deal's margin unconverted, EARN_GROSS recognised in the collection currency.
BALANCE_CONVERSION	Requested (Finance) → Approved (Management) → Booked; Rejected. A banking-ready balance waits for conversion_due_date	Owns exactly one CONVERSION; entitlement re-attributed to the served group on booking.
REROUTE	Requested → Approved → In custody → Forwarded → Completed; Rejected	Phase pre_collection or post_collection; outcome via_own_wallet or refund_sender; forward_full or forward_and_recover_dues.
REROUTE, phase post_conversion	Requested → Approved (Finance, Management) → Reconverted → In custody → Forwarded → Completed; Rejected	Trigger as for any reroute; outcome via_own_wallet only; the reroute owns the balance conversion back and names the second conversion; the leg returns to Converted at the alternate partner. Starts only from a closed payout instruction at the first partner: rejected or acknowledged as not executed, with reference, and the balance reconciled (2d).
EARNINGS_OFFSET	requested (Finance) → approved (Management) → posted	Recovers EARNINGS_RECEIVABLE rows at the from-partner; the swap is recorded for NPL in FX_RESULT.
SETTLEMENT	Prepared → Approved → Released → Completed; Rejected (back to Prepared). A bank-rail run approved after the cutoff is released on execution_due_date	Completed when every line is confirmed, short or returned.
SETTLEMENT_LINE (outcome)	instructed → acknowledged (= sent) → sent → confirmed | short | returned; instructed → outcome unknown → acknowledged | rejected → re-instructed under a new reference, as a retry (2d)	The promise is discharged on release; a short confirmation re-raises it for the missing part and a return for what came back. A line paying a group-less receiver carries deal_id and is never an offset; a hop line posts only when the far side credits. short added.

Entity	States and transitions	Notes
SETTLEMENT_LINE, “sent, not yet confirmed”	sent (released by the partner, released_at set) → confirmed | short | returned	NPL Finance confirmed on 20 October that this state is shown by name on the settlement screens and on the Finance log report of released, unconfirmed lines (Phase 2, M7). A line whose expected_confirmation_by has passed is on the exception list; the promise was discharged on release (Ledger Posting Design, rule 13) and nothing posts until the line ends confirmed, short or returned.
CONFIRMATION (result)	full | short	Full posts nothing; short opens a SHORTFALL unless the treatment is waived.
SHORTFALL	open → topped_up	Top-up funded from the pool, Management approval, paid with the next settlement.
SETTLEMENT_RETURN	recorded (terminal)	Funds a return leg or stays in balance; the bounce fee is a BANK_FEE_EVENT.
BANK_FEE_EVENT	open → applied	Applied to the settlement that carried the true-up; treatment waived / carry_forward / absorbed; a covered fee is a FEE_DECISION.
COLLECTION_RECEIVING_ENDPOINT (status)	inactive → active; active → suspended; a single_use endpoint retires after its one reference	A bank endpoint becomes active only with rail set; a wallet may stay without rail, and its rail, when set, equals its network; the network must be one the partner's vehicle offers.
RECEIVING_ENTITY_ACCOUNT (usage)	standing; single_use → retired after the one payout it was reserved for	.
RATE_COMPARISON	open → closed	One per partner × pair × date; closed at end of day.
PARTNER_RATE_VERSION	active → usable → expired; active → breached → approved → usable, or rejected; active → flagged (spread outside the absolute tolerance, either direction) → usable when Finance confirms it with a reason, or rejected; a flagged version prices nothing and the pair is not frozen (Calculation Specification 2a)	Never overwritten; each conversion names its version; stored as quoted with direction and, for cash, the city.
PARTNER_CUTOFF_OVERRIDE	recorded (one day or a weekday rule)	An agreed extension or an early close; the effective cutoff is stamped on the leg at rate lock and on the settlement at approval.
DESTINATION_APPROVAL (approval_status)	pending_partner → approved → suspended → approved	A quote-first deal leaves Inquiry only if the entitled group has an approved destination with the chosen partner. Payout details follow the account's kind; a wallet account carries its screening.

Entity	States and transitions	Notes
FEE_DECISION	proposed (Finance recommends) → approved (Finance; Management for a covered bank fee or a markup above the cap) → applied; rejected	One subject (deal leg, settlement or balance conversion); standing rules are unaffected.
SENDER_CREDIT	open → applied	Applied to a later deal or refunded (a void disposition); never netted against anything NPL is owed.
EARNINGS_RECEIVABLE	outstanding → recovered (recovered_by_deal_id on a reroute, recovered_by_offset_id on an earnings offset)	Recovered on the partner's fee cycle, by netting on a reroute, or by an earnings offset.
MARKUP_SHARE_ACCRUAL	accrued (with the conversion) → reconciled → settled	Direction payable (NPL pays) or receivable (the party holds the funds and pays NPL the remainder monthly).
PARTNER_REBATE_ACCRUAL	expected (with the conversion, in payout_currency) → reconciled → received	Monthly against the partner's statement, NPLify deals only.
INVOICE (NPL–GR)	open → settled; void. kind vendor: discharged line by line (obligation_discharged on release); kind resale: collected by its deal groups (invoice_amount)	Posted once each in the invoice currency (RESELL_COST / VENDOR_PAYABLE; RESALE_RECEIVABLE / RESELL_REVENUE); the reseller revenue is intact by construction.
LOSS_EVENT	open → approved (Management) → posted	As Figure 6; pool_funded, wallet_funded and funding_reference record how the make-good was funded.
LOSS_RECOVERY	agreed → received (a client share is received at once)	One row per share; each state change posts one transaction under source LossRecovery.
APPROVAL	requested → approved | rejected; requested → escalated → approved | rejected	As Figure 5; the approver's role holds may_approve for the action (ROLE_RIGHT).

7a · The point of no return, and what can still be done after it

Rule 3 of Section 6 says that money which has moved is never rolled back. The table names the moment in each stage after which that holds, and the only recovery actions that remain. The same points are marked on Figures 1 to 4.

Stage	Point of no return	Before it	After it, the only recovery actions
Collection	The sender's money lands on the endpoint (hash confirmed, slip received, cash counted)	Void before money moved: no posting, instructions withdrawn (Operations, Finance)	Post-money void with a disposition: refund to the sender's return receiver, sender credit, or hold (Management); a loss event if the money is lost after verification
Conversion	The partner books the conversion	Conversion instruction withdrawn; deal stays Collected	The booked figure stands; a variance beyond tolerance is a Finance decision and, if needed, a reversing entry; a reroute after conversion (Section 2b) moves the balance, never the booking

Stage	Point of no return	Before it	After it, the only recovery actions
Settlement	The partner releases the payout (execution)	Rejected back to Prepared; lines changed and re-submitted	Nothing is cancelled: the line ends confirmed, short or returned; a return funds a return leg or stays in balance; a duplicate instruction is refused by its idempotency key
External instruction (any)	The partner acknowledges the instruction (2d); the money moves at execution	Withdraw the instruction (Operations, Finance); a second instruction for the same promise is refused while the first is open	Nothing is re-instructed until the partner reports rejected or not executed, with its reference; outcome unknown waits on the partner's statement; a Management exception needs the partner's written confirmation and the reconciliation
Own-wallet transfer	NPL's signers broadcast the transaction	Management rejects; funds stay in custody	The transfer is final on chain; a wrong destination is a loss event; the hop or forward is recorded when the far side credits

8 · Aging and the daily exception list

These things age and are reported daily with deal, reason and hours since the event: a quote awaiting the customer's code (Email OTP), with the hours left in its code window and whether the tripwire has passed; client money in NPL's own wallet (custody — reroutes, bridge collections awaiting their hop, own-desk balances), open offsets across receiver groups, open shortfalls, open variance exceptions, a pool below zero awaiting the next conversion, an unfunded make-good (MAKEGOOD / LOSS_PAYABLE), loss recoveries agreed and not received, and settlement lines sent beyond the rail's expected time, legs on hold (partner or screening) with the hold's age and source, and external instructions whose outcome is unknown. Breached rate versions, frozen partner × pairs and quotes past their validity appear on the same list. Time limits for each are project configuration (THRESHOLD, OWN_WALLET.custody_max_hours) and are not fixed by this document.

8a · Next action and time limits

NPL's decision (19 October): pending items belong to the role, not to a named person. Two things are therefore required of every pending item, and one time limit per stage.

Requirement	What every pending item shows or does
Visible to the whole role	Every pending approval, collection, conversion, settlement or exception is on the list of every user in the role that must act, whoever started it; the initiator's absence changes nothing.
Self-explanatory	Each item shows its state, why it is pending, the last action with its time and user, the next action required, the role that must act, how long it has been pending, and the cutoff it is running against (the per-role lists gain a next-action column in Phase 1).
Time limit per stage	THRESHOLD kind approval_time_limit, one row per stage, in minutes, with an escalation role (Configuration Schema). When the limit passes, the item appears on the escalation role's list as overdue and raises an alert (Phase 3, M8). NPL maintains the values: Finance proposes, Management approves. A cutoff warning is raised a configurable number of minutes before a partner's cutoff for any item still pending against it.

Default time limits (illustrative, from the design review)

Stage	Role that acts	Default limit	Escalates to
Customer approval (Email OTP)	Operations	the tripwire window from the send (default 4 hours): verify with the customer outside the chat; the code window (default 24 hours) ends the item	Finance

Stage	Role that acts	Default limit	Escalates to
Collection verification	Operations	15 minutes from the partner's confirmation	Finance
Partner rate confirmation	Finance	15 minutes from the rate being entered	Management
Conversion approval	Finance	15 minutes from the request	Management, at once if the cutoff is within 30 minutes
Settlement approval	Finance	30 minutes from the request	Management
Partner execution confirmation	Operations	the rail's expected time (Section 8)	Finance; the partner is chased
External instruction, outcome unknown	Operations, then Finance	the partner reference inquiry is raised the same day the acknowledgement time-out passes; Finance obtains the partner's statement	Management, if the partner has not answered by its next statement or within two business days, whichever comes first (2d)
End-of-day reconciliation	Finance	before the day is closed	Management, if a discrepancy stays open
Cutoff warning	the role holding the item	60 minutes before the partner's cutoff; a second warning at 30	Management at the second warning

These are the values proposed in the review; NPL sets the real ones per project before the parallel run. Where the delay affects a client (a missed cutoff, a reroute, a changed settlement date), the Operating Procedures say when and how the client is told.

State names are the proposal for the state columns the ERD leaves as free text; once agreed they become the enumerations of those columns.